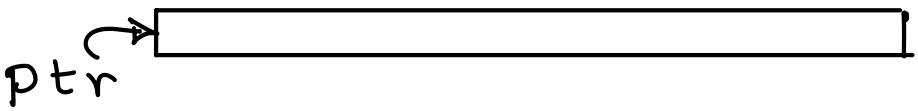
The following data definition statements mean the same thing:

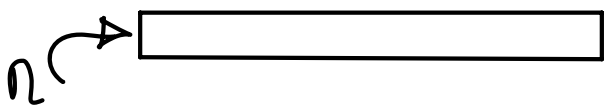
```
int* ptr;
int *ptr;
int * ptr;
int                 *                       ptr;

int*                              ptr;
```

---

int* ptr;



int n;



ptr = &n;

type(n) == int ... assigned by programmer.

type(&n) == int* ... because of &n is an address of which n is a name.

float f_num;

type(f_num) == float ... assigned by programmer

type(&f_num) == float*

struct Date myDate;

type(myDate) == struct Date

type(&myDate) == struct Date*

type(n) == int

int m;

m = n;

type(n) == int & type(m) == int (lhs type == rhs type)

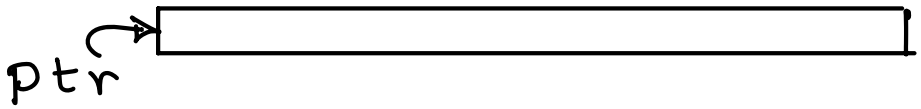
ptr = &n;

type(n) == int ... assigned by programmer
type(&n) == int*
type(ptr) == int* .. assigned by programmer

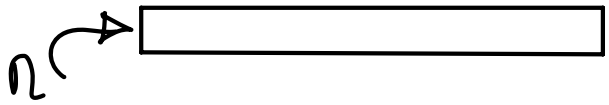
ptr = &n;

---

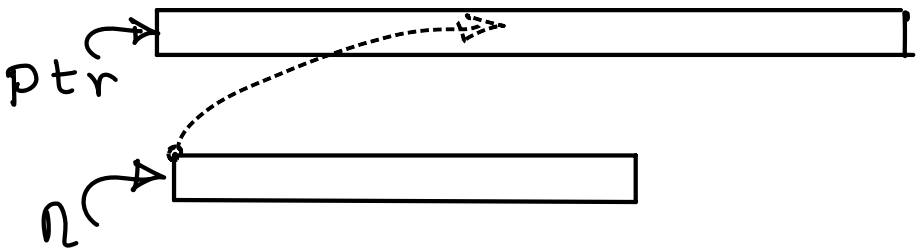
int* ptr;



int n;



ptr = &n;

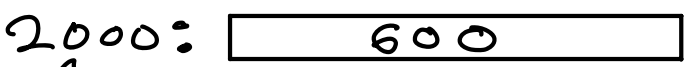


Base address of n is stroed in ptr.

lhs = rhs; // rhs expression is evaluated to a data value
// lhs expression is evaluated to a location value (i.e. address in memory)
// assignment operator says: store data value (which is a result of evaluation of rhs)
// at a location value (which is a result of evaluation of rhs)

int* ptr;

int n;



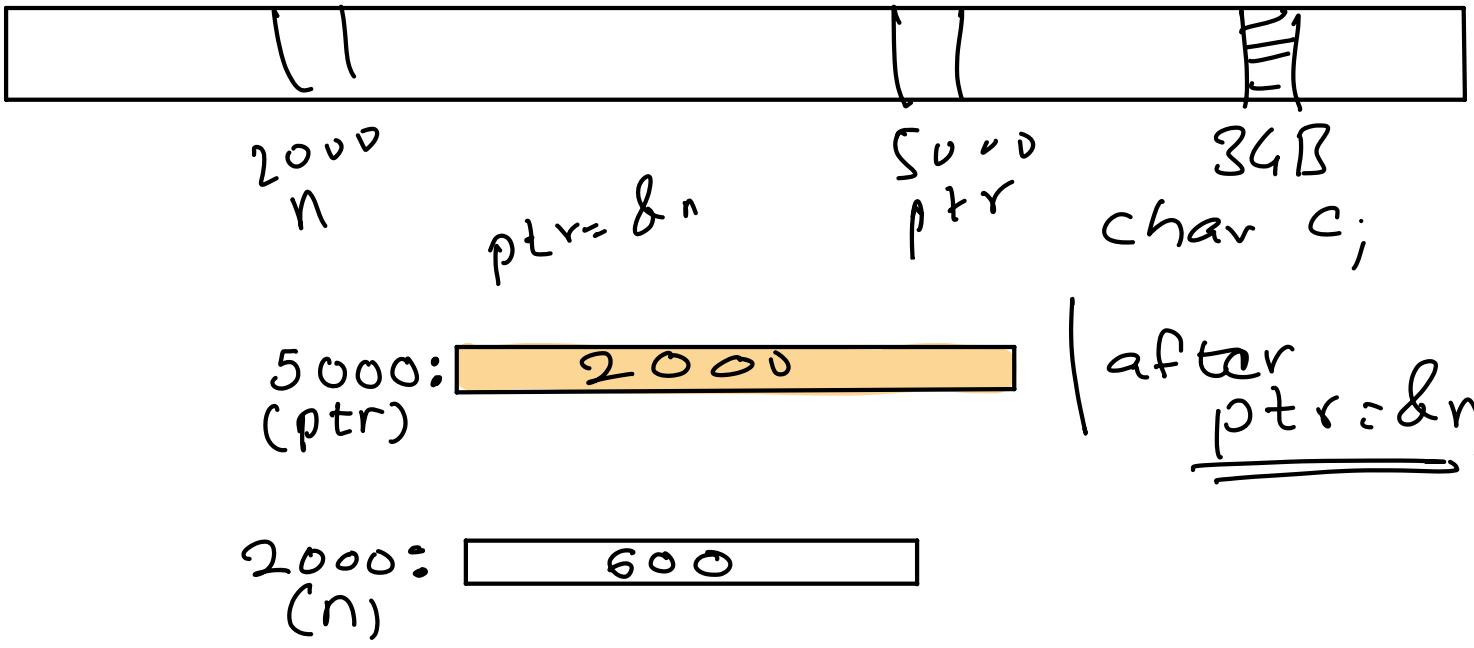
int m = n; [eval: 600]

int* ptr = &n; [eval: 2000]

n = 700;

700 = rhs = 700 = data value.

n = lhs = M[2000] = location value.



---

ptr = 700;

*ptr = 700;

rhs = 700 = rhs value 700

Case 1: ptr = 700;

LHS expression = ptr
after evaluation M[5000] will be the location value.
Therefore, 700 will be stored at memory location starting from M[5000].
i.e. 2000 which is a base address of integer n, which was stroed at M[5000] will be ERASED and replaced by 700 (which may not be base address of any integer)

Case 2:

*ptr = 700;
rhs = 700
after evaluation : integer 700 will be an evaluation of the rhs expression.
rhs value = 700

Lets find out a location value for 700.

lhs expression == *ptr
CONTENT stored INSIDE ptr is TAKEN as a LOCATION VALUE instead of
LOCATION WHOSE NAME IS ptr.

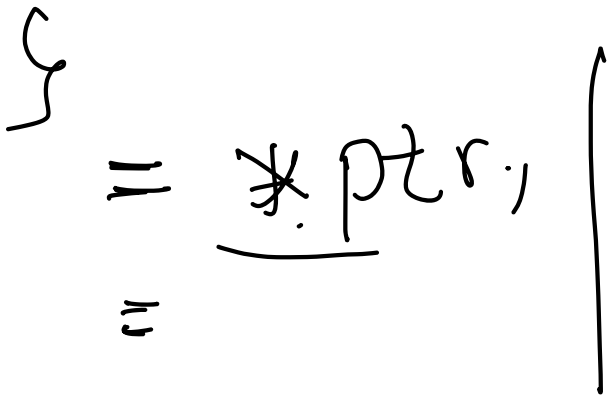
In out case ptr is a name of M[5000]
2000 is a CONTENT stored at M[5000]

*ptr is a LEFT HAND SIDE EXPRESSION
CONTENT STORED INSIDE ptr == 2000
is a LOCATION VALUE, instead of LOCATION 5000 (whose name is ptr)

700 will be stored at M[2000] which is nothing but base address of n.
Therefore, we will get effect of erasing 600 of n and replacing it with 700.

```
int n;
n = 10;

int* ptr;
ptr = &n;

int n = 10;
int* ptr = &n;
```

```
void f(void)
{
    int * ptr;
    ptr = &n;
    ptr = 1547;
    *ptr;
```



ptr = 2000

*ptr M[2000]

r/w

int *ptr

*ptr

M[2000:2003]

R or W

Short *ptr.

*ptr

M[2000:2001]

r/w

long long int *ptr.

*ptr

M[2000:2007]

T* ptr;

lhs = *ptr;
*ptr he expression rhs la theun, C programmer, C compiler la, ase sangat asto, ki
ptr hya variable madhali value address mhanun treat kar ani tya address pasun sizeof(ptr type) evadhe bytes read kar

*ptr = rhs;
*ptr he expression lhs la theun, C programmer, C compiler la ase sangat asto ki,
ptr hya variable madhali value address mhanun treat kar ani tya address pasun sizeof(ptr type) evadhya bytes var rhs_value write kar.

lhs = *ptr;
By writing *ptr expression on rhs, a C programmer asks the C compiler to treat value in ptr as an address and read sizeof(ptr type) bytes from that address.

*ptr = rhs;
By writing *ptr expression on lhs, a C programmer asks the C compiler to treat value in ptr as an address and write rhs_value on sizeof(ptr type) bytes from that address.

CAREER DEFINING.